

Client Precondition Report

GlamBrandi Travel & Shopping (GBTS)

CS 472, Fall 2026

Team assumption for this report: 8–12 senior Computer Science students, each contributing about 120 hours over the semester (roughly 960–1,440 total team hours) across about 13 weeks of active development. Students bring strong general programming and Git/GitHub skills but are not expected to arrive knowing the specific frameworks below. Every specialized tool is listed as a skill to be learned, not a prerequisite.

1. Company Information

Company Name: Innovative Configuration International LLC

Primary Contact Person: Jay Karma

Email Address: jay.karma@innovativeconfiguration.com; sameer.dingore@innovativeconfiguration.com

Website: www.innovativeconfiguration.com

Location / Time Zone: Aptos, CA (Pacific Time)

2. Team Structure & Required Skills

a. Team Composition

We picture one team of 8–12 students split into small feature squads, so everyone owns a real part of the product:

- **Rotating Team Lead / Scrum facilitator:** one student, rotating every 2–3 weeks so several students gain the experience
- **Frontend squad (3–4):** search UI, map view, detail pages, and the chat interface
- **Backend squad (3–4):** REST API, data model, third-party API integrations, and search indexing
- **Data & AI squad (2–3):** catalog data, the AI chat assistant, and (stretch) deals and discounts

Collaboration expectations:

- Two-week sprints tracked on a shared GitHub Projects board
- Short async standups 2–3x per week plus one live weekly sync
- All code merges via pull request with at least one peer review
- Squads are not silos. We encourage cross-squad pairing and rotating people through areas, so everyone leaves with broad full-stack exposure

b. Required Baseline Skills

- Comfortable programming in at least one language (JavaScript or TypeScript a plus, not required)
- Basic understanding of how web apps work (client/server, HTTP, JSON)
- Familiarity with Git and GitHub (branches, pull requests)
- Basic grasp of software development practices (testing, code review)
- Willingness and enthusiasm to learn new frameworks and tools

No prior experience with any specific framework, cloud service, or AI tooling is required.

c. Skills to Be Developed During the Project

- Modern full-stack web development with Next.js / React and TypeScript
- Building REST APIs with NestJS
- Relational data modeling with PostgreSQL and Prisma
- Working with real third-party APIs (maps, places, events, weather)
- Full-text and faceted search with Meilisearch or Typesense
- Interactive maps with Mapbox

- Integrating an AI assistant into a product (prompting, streaming responses)
- Testing, CI/CD with GitHub Actions, and a code-review workflow
- Agile teamwork, sprint planning, and shipping a real consumer product

3. Project Title & Acronym

Project Title: GlamBrandi Travel & Shopping

Acronym (GitHub repository): GBTS

4. Project Overview & Problem Statement

The problem: Las Vegas has an overwhelming amount to offer (restaurants, shows, spas, tours, shops, and attractions), but visitors and locals struggle to discover the right things for them and to find good deals. The information is scattered across dozens of separate websites and apps.

GlamBrandi is a consumer web app that brings all of this into one place. Users can search for things to do and buy, explore results on a map, open detail pages, and chat with an AI assistant that gives personalized, conversational recommendations. For example: “Find me a fun evening for two under \$150 near the Strip.” A stretch goal is location-based discounts, including special perks for verified local residents.

We group everything users can discover into four buckets we call PSAA: Products, Services, Activities, and Attractions.

Over time, GlamBrandi should also become a place where visitors and locals share what they love about Las Vegas: tips, favorites, and firsthand recommendations. The goal is a community around Vegas discovery, not just a listings directory.

- **Target users:** Las Vegas visitors planning a trip, and locals looking for things to do and deals near them
- **Goals & impact:** give users a fast, easy way to discover Vegas experiences, grow an engaged community around them, and give the business a discovery platform that stands out
- **What students gain:** the experience of shipping a complete, production-style consumer web app (frontend, backend, data, search, maps, and AI) as a portfolio-worthy team project

5. Solution Overview & Core System Components

The system is a full-stack web application: a Next.js frontend, a NestJS backend API, a PostgreSQL database, and a search engine, plus integrations for mapping, places and events data, weather, and an AI assistant.

Core MVP Deliverables (achievable in the 13-week window)

- A browsable, searchable catalog of Las Vegas PSAA items, seeded from a curated dataset and official APIs
- Search with keyword queries and basic filters (category, price range, area)
- Interactive map view of results with markers and popups (Mapbox)
- Detail pages for each item
- An AI chat assistant that answers natural-language questions and recommends items from the catalog
- Responsive, polished UI built with the project’s component library
- Automated tests and a working CI pipeline

Stretch Goals (optional, if time allows)

- User accounts with profiles, saved items, and shareable curated lists
- Community and content features: letting users contribute short tips, write-ups, or guides about the places and experiences they love, and share them with others
- Location-based deals and discounts surfaced on relevant items

- “Local resident” verification to unlock resident-only perks (via a hosted identity provider so students never store sensitive ID data)
- On-demand data enrichment: when a search returns thin results, fetch fresh details from official APIs in the background
- Weather-aware recommendations (for example, suggesting indoor options on very hot days)

6. Technical Considerations

Preferred tech stack (skills to learn, not prerequisites)

- **Frontend:** Next.js (App Router), React, TypeScript, TanStack Query, a shared component library, Mapbox GL JS
- **Backend:** NestJS, TypeScript, Prisma ORM
- **Database:** PostgreSQL
- **Search:** Meilisearch or Typesense
- **AI:** an LLM provider accessed through a thin backend proxy, so API keys stay server-side
- **Optional / stretch:** Redis and a job queue (BullMQ) for background data enrichment

APIs, datasets & tools

- Google Places/Maps, Yelp Fusion, Eventbrite, and LVCVA (Las Vegas tourism) data for catalog content
- Open-Meteo for weather (no API key required)
- Mapbox for maps
- We will provide sandbox and test API keys where needed

Constraints & integration

- **API-first:** where a source offers an official API, use it, and respect each provider’s terms of service and rate limits. We are not asking students to build web scrapers
- **Performance:** the app should stay responsive on both mobile and desktop
- **Structure:** a monorepo with a clear frontend/backend split. The frontend talks to the backend over REST/JSON, and the backend brokers all third-party and AI calls

Quality, testing & CI

- Unit and integration tests (Jest)
- GitHub Actions CI: lint, type-check, and tests on every pull request
- Peer review required before merge; preview deployments encouraged

Security & privacy

- No secrets in client code. All keys and tokens live server-side via environment variables
- Collect the minimum personal data. If the identity-verification stretch goal is pursued, use a hosted provider (such as Stripe Identity) so the team never stores raw ID documents

6.1 Generative AI Tool Policy

- **Status:** permitted and encouraged. We use these tools ourselves and want students comfortable with an AI-assisted workflow
- **Approved tools:** GitHub Copilot, Cursor, Claude / Claude Code, ChatGPT, Codex. Use whichever you prefer
- **Do not share with external AI tools:** credentials, API keys, or tokens; any real user or customer personal data; and any business data we mark confidential. Project source code may be used with these tools for this student project
- **Disclosure:** no line-by-line attribution required, but please note in the PR description when a change was substantially AI-generated. Make sure every author can explain and defend the code they submit; understanding your code matters more than how it was written

7. Innovation & Competitive Advantage

Most discovery sites are static directories. GlamBrandi pulls several things together in one place: unified search across products, services, activities, and attractions; an interactive map; a conversational AI assistant that gives personalized recommendations; and location-based local discounts. The Las Vegas focus, fresh data from official sources, and resident-only perks set it apart from generic travel apps.

8. Proposed Implementation Timeline (13 Weeks)

- **Weeks 1–2:** Onboarding and foundations. Environment setup, monorepo scaffolding, CI pipeline, architecture walkthrough, and forming squads. Deliverable: a running skeleton app on a preview URL with green CI
- **Weeks 3–4:** Data and search core. Data model and Prisma schema, seed the PSAA catalog, backend search API, and a basic search UI. Deliverable: users can search the seeded catalog
- **Weeks 5–6:** Map and detail pages. Mapbox integration, result markers and popups, filters, and item detail pages. Deliverable: map-based browsing works end to end
- **Weeks 7–8:** AI assistant. Backend AI proxy with streaming, chat UI, and responses grounded in catalog data. Deliverable: a working assistant that recommends real items
- **Week 9:** Integration and midpoint demo. Connect the pieces, fix rough edges, run an accessibility and performance pass, and add tests. Deliverable: a cohesive MVP demo
- **Weeks 10–11:** Stretch goal. The team picks one: deals and discounts, on-demand enrichment, or resident verification. Deliverable: one stretch feature integrated
- **Week 12:** Hardening. Expand tests, docs, and performance work, plus a bug bash. Deliverable: release-candidate quality
- **Week 13:** Final polish and handoff. Deployment, handoff docs, and the final presentation. Deliverable: deployed app, documentation, and a demo

9. Deployment & Support Expectations

- **Hosting / infrastructure:** frontend on Vercel; backend on a student-friendly container host (Render, Railway, or Fly.io); managed PostgreSQL (Neon, Supabase, or Railway); search engine on a small managed or self-hosted instance. Free or low-cost tiers are fine for the course
- **We will provide:** repo access, the shared component library, sandbox and test API keys, seed data, and a Mapbox token
- **Documentation / training:** a clear README and setup guide, brief API docs, and a final handoff document (architecture overview plus how to run and deploy). We will run an onboarding session in week 1 and be available weekly

10. Confidentiality & Intellectual Property

Every student on the project is required to sign a Non-Disclosure Agreement (NDA) before work begins. This is a firm requirement, not optional. Even so, we still want you to be able to show off what you build. Specifically:

- All team members sign an NDA at the start of the project
- You may still include your capstone work in your portfolio: demo it, describe what you built, and show screenshots and the code you personally wrote
- Keep confidential the things the NDA covers: API keys and credentials, any real user or customer data, and any unreleased business plans or internal information we share with you
- Do not publish proprietary datasets we provide; check with us first
- The company owns the code and product delivered, while you keep the right to showcase your contributions and talk about the technologies and skills you used

11. Proposed Success Metrics

Quantitative

- All Core MVP deliverables complete and ready to demo
- Catalog seeded with a meaningful number of Vegas items (target 300 or more)
- Search returns relevant results quickly (target under about 500ms for typical queries)
- AI assistant returns useful, on-catalog recommendations for a set of test prompts (target 80% or more judged helpful)
- Green CI on the main branch and the agreed test-coverage target met
- Solid performance and accessibility scores on key pages

Qualitative

- Code quality, clarity, and sensible architecture
- UX polish and responsiveness
- Documentation quality and ease of running the project
- Team collaboration, communication, and PR-review culture
- Quality of the final demo and handoff

12. Mentorship & Communication Plan

- **Cadence:** a weekly live meeting (matching the professor's preference), plus async check-ins
- **Platforms:** Slack for day-to-day chat, GitHub for code and issues, and video for the weekly sync
- **Response time:** within about one business day on Slack
- **Mentors:** Sameer Dingore (SWE Tech Lead), Heath Mensch (Product Engineer, AI Engineering team), and Daniel Lesinsky (Product Engineer, Frontend team) will mentor the team across backend, AI, and frontend work

13. Additional Notes

- This is a real product. Strong work may make it into production
- Our product direction leans toward community and user-generated content, so students interested in social, content, and sharing features will find room to grow into that as the project develops
- We favor an API-first, terms-respecting approach to data; no web scraping is required
- Come with questions. We would rather clarify early than have you guess